알약 객체 탐지 최종 보고서

(부제 : 어플 이게머약💊)

코드잇 *AI*엔지니어 *10*기_*4*팀

팀원 : 김재헌A, 김재헌B, 임유빈, 김건오

목차

1. 서론 **2**
1.1 배경 및 목적 2
2. 데이터셋 구성 및 전처리 **3**
2.1 데이터 출처 3
2.2 데이터셋 구성 3
2.3 데이터 전처리 5
3. 모델 후보 선정 및 실험 설계 **6**
3.1 후보 모델 목록 및 선정 근거 6
3.2 공통 실험 환경 7
3.3 평가 지표 7
4. 모델 비교 실험 결과 **7**
4.1 전체 모델 성능 비교 8
4.2 결과 분석 8
5. 선정 모델 고도화 **8**
5.1 데이터 증강 9
5.2 하이퍼 파라미터 튜닝 9
5.3 데이터 추가 12
6. UI 디자인 및 구현 **13**
6.1 스플래시 스크린 13
6.2 홈 스크린 13
6.3 알약 리스트 스크린 14
6.4 복약 알림 관리 14
6.5 환경설정 15

1. 서론

1.1 배경 및 목적

- 본 프로젝트는 이미지 1장에 3~4개의 알약이 포함된 환경에서 각 알약의 위치(bounding box)와 클래스명을 정확하게 예측하는 객체 탐지(Object Detection) 모델을 개발해 내는 것을 목표로 한다.

- 추가적으로, 해당 모델을 활용하여 실생활에 사용할수 있을 만큼의 어플 데모를 만드는 것이 목표이고, 데이터 증강 및 하이퍼 파라미터 튜닝을 통해 모델 성능을 고도화하여 신뢰도 있는 모델을 활용하는 것이 최종 목표이다.

- 데이터셋은 코드잇(Codeit)에서 제공한 기본 데이터와 직접 수집한 AI 허브의 추가 데이터를 병합하여 구성하였으며, 총 118종의 알약 클래스를 대상으로 한다. 핵심 평가 지표는 mAP@75-95이며, 목표 성능은 0.95 이상이다.

2. 데이터셋 구성 및 전처리

2.1 데이터 출처

본 프로젝트의 학습 데이터는 두 가지 소스를 병합하여 구성하였다

- 코드잇(codeit)에서 제공한 기본 데이터셋
- 추가 데이터셋(출처: AI허브)

2.2 데이터셋 구성

- 기존 Codeit에서 제공받은 데이터셋은 총 56종의 알약 클래스로 구성되어 있었으며, 클래스별 데이터 분포에 큰 편차가 존재하였다. EDA(탐색적 데이터 분석)를 수행한 결과, 일부 클래스는 150장 이상의 이미지를 포함하고 있었던 반면, 일부 클래스는 3장 이하의 이미지로만 구성되어 있어 심각한 클래스 불균형 문제가 확인되었다. 이러한 불균형은 객체 탐지 모델 학습 과정에서 특정 클래스에 대한 학습

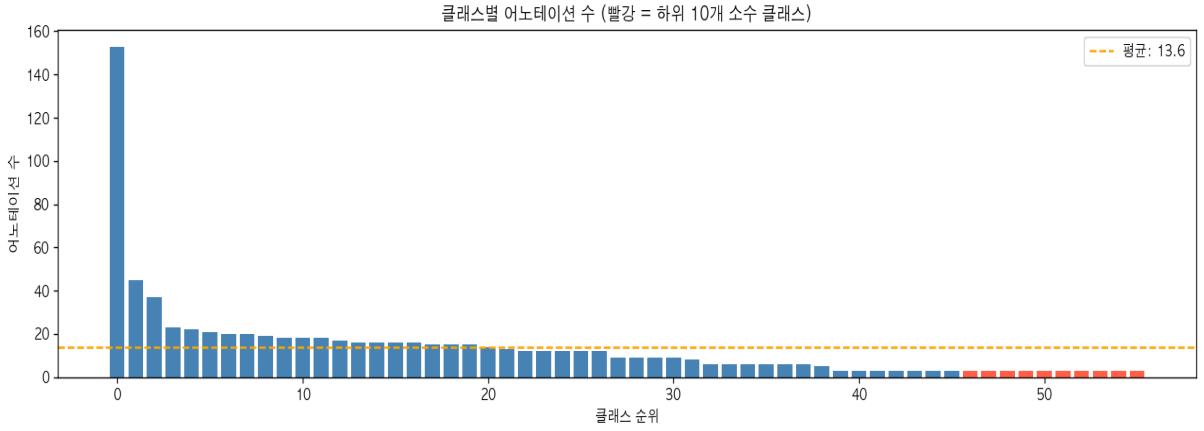
편향을 유발할 수 있으며, 데이터 수가 적은 클래스의 경우 탐지 성능 저하 및 재현율 감소로 이어질 가능성이 높다.

- 또한 데이터셋 분석 과정에서 배경색, 촬영 환경, 알약의 크기 및 형태, 촬영 각도, 약 고유 식별값 등의 분포를 함께 확인하였다. 분석 결과, 대부분의 이미지가 유사한 배경환경에서 촬영되어 데이터 다양성이 제한적이었으며, 클래스별 샘플 수 차이 역시 매우 크게 나타났다. 이는 모델이 특정 환경에 과적합될 가능성을 높이고, 실제 다양한 환경에서의 일반화 성능을 저하시킬 수 있는 요소로 판단하였다.

- 데이터 전처리 과정에서는 여러 개로 분산되어 있던 어노테이션 파일을 이미지 한 장당 하나의 어노테이션 파일로 병합하여 일대일 매핑 구조로 재구성하였다. 이후 객체 탐지 학습에 적합한 YOLO형식으로 라벨 데이터를 변환하고, 학습 및 검증 데이터셋으로 분리하였다.

- 기존 데이터셋의 한계를 보완하기 위해 AI Hub의 추가 알약 이미지 데이터셋을 활용하였다. 추가 데이터셋에는 기존 데이터 대비 다양한 촬영 환경과 추가 클래스 정보가 포함되어 있었으며, 이를 기존 데이터셋과 통합하여 부족한 클래스의 데이터를 보강하였다. 이를 통해 클래스 불균형 문제를 완화하고, 다양한 환경에 대한 데이터 다양성을 확보하여 모델의 일반화 성능 향상을 목표로 하였다.

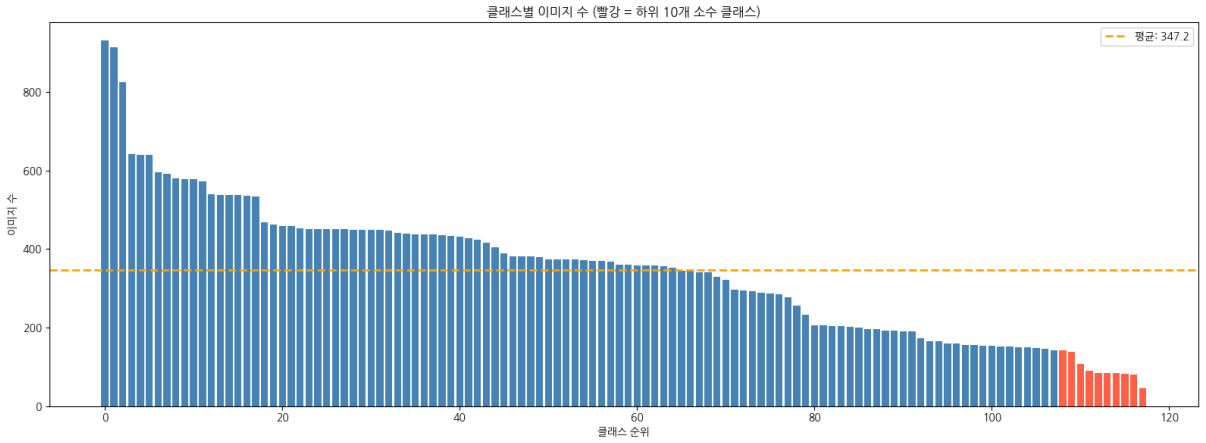
*기존 데이터 클래스 이미지 분포도



상위 **5**개 클래스 **(**데이터 많음**)**		하위 **5**개 클래스 **(**데이터 부족**)**	
일양하이트린정 2mg	153	맥시부펜이알정 300mg	3
기넥신에프정(은행엽엑스)(수출용)	45	써스펜8시간이알서방정 650mg	3
아토젯정 10/40mg	37	동아가바펜틴정 800mg	3

크레스토정 20mg	23	트루비타정 60mg/병	3
아토르바정 10mg	22	라비에트정 20mg	3

*데이터 추가 후 클래스 이미지 분포도



상위 **5**개 클래스		하위 **5**개 클래스	
가바토파정	930	토비애즈서방정 4mg	84
동아가바펜틴정 800mg	914	플리바스정 50mg	84
리피토정 20mg	824	트루패스정 4mg	83
아스피린프로텍트정 100mg	641	베시케어정 10mg	81
동아오팔몬정 (리마프로스트알파-시클로덱스트린포접화합물)	640	기넥신에프정 (은행엽엑스) (수출용)	45

2.3 데이터 전처리

| yolo 계열 모델

YOLO 계열 모델이 학습할 수 있도록 기존 JSON 형식의 어노테이션 파일을 YOLO 학습 형식에 맞는 TXT 파일로 변환하였다.

YOLO 계열 모델은 학습 루프 내부에서 전처리가 자동 처리되며, 다음 네 단계로 구성된다.

- 리사이즈: Letterbox 방식으로 640×640 해상도로 변환. 원본 비율을 유지하며 빈 영역은 회색(픽셀값 114)으로 패딩 처리

- BGR → RGB 변환: OpenCV로 읽어들인 BGR 포맷 이미지를 RGB로 변환
- 정규화: 픽셀값을 255로 나눠 0.0–1.0 범위로 스케일링
- 채널 축 변환: HWC(높이·너비·채널) 포맷을 CHW(채널·높이·너비) 텐서 포맷으로 변환

| torchvision 계열 모델**(Faster-R-CNN, DETR, SSD**등**)**

torchvision 기반 모델에는 basic_transform()을 통해 두 단계의 전처리를 적용하였다. 먼저 ToTensor()로 PIL 이미지를 텐서로 변환하여 픽셀값을 0–255에서 0.0–1.0 범위로 스케일링하고, 이후 Normalize()로 ImageNet 통계값 기준 정규화를 수행하였다.

3. 모델 후보 선정 및 실험 설계

3.1 후보 모델 목록 및 선정 근거

| Two-stage 계열

Faster R-CNN (ResNet 백본*)* : Region Proposal Network(RPN)을 통해 후보 영역을 먼저 추출한 뒤 분류를 수행하는 two-stage 구조로, 소형 객체가 밀집된 환경에서 높은 탐지 정밀도를 보이는 대표 모델이다. 알약처럼 크기가 작고 클래스 간 형태 차이가 미세한 태스크에서의 성능을 검증하기 위한 기준 모델로 포함하였다.

Faster R-CNN + MobileNet : 동일한 two-stage 구조에서 ResNet 백본을 경량 아키텍처인 MobileNet으로 대체한 모델이다. 정확도를 일부 희생하더라도 추론 속도와 파라미터 수에서 이점이 있는지 비교하기 위해 선정하였으며, 향후 모바일·엣지 환경 배포 가능성을 염두에 두었다.

| One-stage 계열

RetinaNet : Focal Loss를 도입하여 클래스 불균형 문제를 구조적으로 완화한 one-stage 모델이다. 본 데이터셋은 클래스별 이미지 수 편차가 크게 나타나는데,

RetinaNet의 Focal Loss가 소수 클래스 학습에 미치는 효과를 실험적으로 확인하기 위해 선정하였다.

SSD (Single Shot MultiBox Detector) : 다중 스케일의 feature map에서 동시에 탐지를 수행하는 경량 one-stage 모델이다. 추론 속도가 빠른 대신 소형 객체 탐지 성능이 상대적으로 낮다고 알려져 있어, 다른 모델들과의 정확도-속도 트레이드오프 비교를 위한 하한선(baseline) 모델로 포함하였다.

DETR (Detection Transformer) : CNN 기반 모델과 달리 Transformer의 self-attention 메커니즘을 활용하여 객체 간 전역적 관계를 학습하는 end-to-end 탐지 모델이다. 기존 CNN 기반 모델 대비 이 태스크에서 Transformer 구조가 유효한지 검토하기 위해 실험 대상에 포함하였다. 단, DETR은 수렴에 많은 에폭이 필요한 것으로 알려져 있어 100 에폭 환경에서의 성능 한계도 함께 확인하고자 하였다.

| YOLO 계열

YOLOv8n : 안정적인 성능과 광범위한 커뮤니티 검증을 바탕으로 YOLO 계열의 기준 모델로 선정하였다. 경량(nano) 변형으로 최소한의 리소스에서 달성 가능한 성능 하한을 확인하는 역할을 한다.

YOLOv11s / YOLOv11m : 최신 YOLO 아키텍처의 소형(small)·중형(medium) 변형을 함께 실험하여 모델 크기 증가에 따른 성능 향상 폭을 비교하였다. 파라미터 수와 정확도 간의 트레이드오프를 정량적으로 확인하기 위한 목적이다.

YOLOv26n / YOLOv26s : 추가적인 최신 경량 아키텍처로, nano·small 두 변형을 동시에 실험하여 YOLOv11 계열과의 세대 간 성능 차이 및 경량화 효율을 비교하기 위해 선정하였다.

3.2 공통 실험 환경

항목	설정값
GPU	google colab T4 GPU
batch size	Auto(-1)
Epoch	100
Optimizer	AdamW

3.3 평가 지표

- mAP@50 : IoU threshold 0.5 기준 평균 정밀도

- mAP@ 75-95 : IoU threshold 0.75~0.95 기준 평균 정밀도

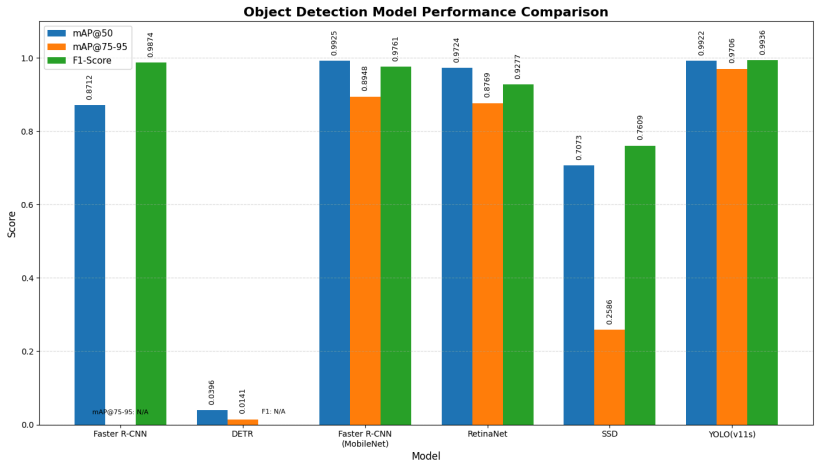
- F1-Score : Precision과 Recall의 조화 평균

4. 모델 비교 실험 결과

4.1 전체 모델 성능 비교

아래 표는 동일한 실험 환경에서 6개 모델의 주요 성능 지표를 비교한 표이다.

모델	mAP@50	mAp@75-95	F1-Score
Faster R-CNN	0.8712	-	0.9874
DETR	0.0396	0.0141	-
Faster R-CNN (MobilNet)	0.9925	0.8948	0.9761
RetinaNet	0.9724	0.8769	0.9277
SSD	0.7073	0.2586	0.7609
YOLO(v11s)	0.9922	0.9706	0.9936



4.2 결과 분석

베이스라인 성능지표를 분석해본 결과, Faster R-CNN 계열의 모델과 YOLO 계열의 모델이 높은 성능지표를 보여 두 모델을 고민했으나, Faster R-CNN의 경우 한번 학습에 너무 오랜시간이 걸려 튜닝 시간을 고려해 **YOLO**모델을 튜닝하기로 결정했다.

5. 선정 모델 고도화

5.1 데이터 증강

| 최종 증강 적용 기법

1. mosaic = 0.0(비활성화) : 여러 이미지를 합치는 mosaic 증강을 끔. 이미 한 이미지에 여러개의 알약이 있어 이를 활성화 하면 알약의 크기가 너무 작아져 성능이 저하된다고 판단
2. flipud = 0.5 : 이미지를 상하로 뒤집어 촬영 각도 다양성 확보
3. scale = 0.1 : 객체 크기를 소폭 축소하여 다양한 거리의 알약 환경
4. hsv_v = 0.5 : 명도(밝기) 변화 폭을 키워 조명 환경 다양성 확보

※ 언급하지 않은 파라미터는 default로 설정된 값을 사용하였을 때 성능이 가장 좋게 측정

| 핵심 지표 변화

구분	mAP@50	mAP@75-95	F1-Score
default	0.8393	0.8063	0.7721
실험 19 (최종)	**0.9670**	**0.9265**	**0.8092**
향상	**+0.1277**	**+0.1202**	**+0.0371**

-> 핵심 지표인 mAP@75-95 기준으로 default 대비 **+0.1202** 향상되었다

5.2 하이퍼 파라미터 튜닝

| YOLO v11s

실험	배치	크기	lr0	lrf	optimizer	mAP50	mAP@75-95	F1
1	13	640	0.000167	기본	AdamW	**0.9922**	**0.9706**	**0.9936**
2	24	640	0.01	기본	AdamW	0.9922	0.9670	0.9936
3	24	640	0.000167	기본	AdamW	0.9922	0.9644	0.9904
4	13	800	0.01	기본	AdamW	0.9896	0.9665	0.9904
5	13	640	0.01	0.1	AdamW	0.9922	0.9682	0.9936
6	13	640	0.000167	0.1	AdamW	0.9922	0.9580	0.9936
7	13	640	0.01	0.01	SGD	0.9921	0.9627	0.9904

mAP@50과 F1-Score는 대부분의 실험에서 0.9922, 0.9936으로 수렴하여 큰 변동이 없었다. 핵심 지표인 mAP@75-95에서는 초기 설정인 실험 1(배치 13, lr0=0.000167, AdamW)이 0.9706으로 가장 높은 성능을 기록하였다.

배치 크기를 24로 늘린 실험 2·3에서는 학습 시간이 약 4분 단축되었으나 mAP@75-95가 소폭 하락하였다. 이미지 크기를 800으로 확대한 실험 4는 학습 시간이 증가하면서도 전 지표가 낮아져 이미지 크기 확대가 본 태스크에 불리함을 확인하였다. lrf=0.1을 적용한 실험 5는 실험 1에 근접한 성능(0.9682)을 보였으나 이를 초과하지는 못하였다. 옵티마이저를 SGD로 교체한 실험 7도 AdamW 대비 유의미한 개선이 나타나지 않았다.

최종 선정 : 실험 **1** — mAP@75-95 **0.9706**

| YOLO v11m

실험	배치	에폭	추가 데이터	mAP@50	mAP@75-95	F1
1	Auto	100	X	0.3667	0.2972	0.6026
2	20	200	X	**0.9870**	**0.9539**	**0.9904**
3	16	100	O	0.9521	0.9030	0.9697

4	16	200	O	0.9664	0.9319	0.9696

AutoBatch가 배치 크기를 6으로 설정한 실험 1은 GPU 메모리를 충분히 활용하지 못하여 mAP@75-95 0.2972에 그쳤다. 배치 크기를 20으로 수동 설정하고 에폭을 200으로 늘린 실험 2에서 mAP@75-95 0.9539로 큰 폭의 성능 향상이 확인되었으며, 이는 초기 실험의 저성능이 모델 자체의 한계가 아닌 에폭 부족 및 배치 설정 오류에 기인한 것임을 시사한다.

추가 데이터를 투입한 실험 3·4는 에폭을 200으로 늘려도 실험 2 대비 mAP@75-95가 낮아 추가 데이터가 오히려 학습 난이도를 높인 것으로 분석된다. 데이터 수 대비 모델 용량이 과도하여 s 계열이 본 데이터셋에 더 적합할 수 있음을 확인하였다.

최종 선정: 실험 **2** — mAP@75-95 **0.9539**

| YOLO v26s

실험	에폭	lr	optimizer	증강	mAP@50	mAP@75-95	F1
1	100	0.01	AdamW	X	0.9529	0.8851	0.9351
2	120	0.01	AdamW	X	0.9051	0.8579	0.8993
3	100	0.01	AdamW	O	0.9169	0.8813	0.8859
4	100	0.01	AdamW	X	0.9739	0.9139	0.9666
5	100	0.01	AdamW	O	0.9456	0.8609	0.9545
6	150	0.01	AdamW	X	0.9273	**0.9418**	0.9765

실험 1은 기본값으로 설정한 베이스라인 모델로, 좋은 성능지표를 기록하였다. 실험 2에서는 에폭을 120으로 늘렸고, 실험3에서는 이미지 크기를 640에서 768로 확대하였으나, 베이스라인 성능에 미치지 못했다. 실험 4에서는 추가 이미지를 학습 데이터에 포함하고 실험 1과 동일한 조건으로 학습한 결과, 전 지표에서 큰 폭의 성능 향상을 기록하였다. 이는 데이터 양 확보가 하이퍼파라미터 조정보다 성능에 더 직접적인 영향을 미침을 보여준다. 실험 5에서는 증강 기법을 적용하였으나, 실험 4 대비 성능이 하락하였다. 실험 6에서는 실험 4의 조건을 유지한 채 에폭을 150으로 늘린 결과, 모든 실험 중 가장 우수한 성능을 기록하였다.

최종 선정: 실험 **6** — mAP@75-95 **0.9418**

| YOLO v26n

실험	에폭	추가 데이터	증강	mAP@50	mAP@75-95	F1
1	100	X	X	0.7118	0.6862	0.7826
2	100	X	O	0.6294	0.5747	0.7396
3	100	X	X	0.7415	0.7094	0.7634
4	100	X	O	0.7828	0.7505	0.8028
5	120	X	X	0.8282	0.7999	0.8000
6	120	X	X	0.8282	0.7999	0.8000
7	120	X	X	**0.9161**	**0.8823**	**0.8803**
8	120	X	X	0.8346	0.8064	0.8138

실험 1~2에서는 기본 설정 대비 이미지 크기를 640에서 768로 확대하고 약한 증강을 추가하였으나, 오히려 성능이 하락하였다. 실험 3에서는 증강을 제외하고 이미지 크기 768을 유지한 결과, 성능지표가 회복되었다. 실험 4에서는 아주 약한 색상 증강만을 추가하여 전반적인 성능 개선을 달성하였다. 실험 5~6에서는 에폭을 120으로 늘리고 학습률을 0.005로 미세 조정하였으나,두 실험 모두 동일한 수치를 기록하며 유의미한 차이를 보이지 않았다. 실험 7에서는 옵티마이저를 AdamW에서 SGD로 수동 고정하고 학습률을 0.005로 직접 제어하였다. 그 결과, 성능지표가 전 지표에 걸쳐 큰 폭의 성능 향상을 기록하였다. 실험 8에서는 학습률을 0.003으로 조정하였으나, 이전 실험보다 성능 저하를 확인하였다.

최종 선정: 실험 **7** — mAP@75-95 **0.8823**

5.3 데이터 추가

데이터를 추가 하기 전, YOLO 모델을 튜닝하면서 약 230장의 데이터로는 더 높은 성능지표를 얻기 어렵다고 느꼈고, AI허브의 다른 경구약제조합 데이터를 활용하기로 결정했다. 이를 이용하게 되면, 클래스 수가 추가되어 목표로 했던 약 어플에 더 걸맞게 많은 약들을 인식할 수 있을 것이고, EDA에서 문제가 되었던 데이터가 부족했던 클래스에 대해서 추가적인 데이터를 확보할 수 있게 되어서 성능지표에 긍정적인 영향을 끼칠 수 있다고 생각해 데이터 추가를 결정했다.

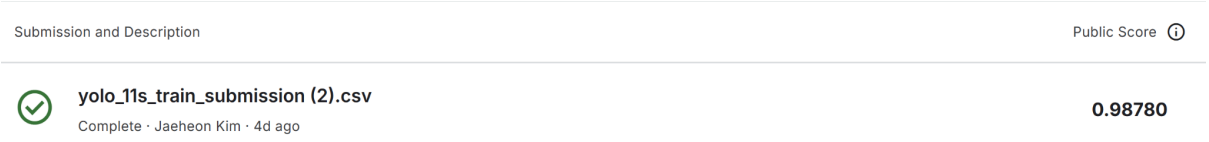
데이터 추가 전/후	총 이미지 수	클래스 수	mAP@50	mAP@75:95	F1 Score

추가 전	234	56	**0.9922**	0.9706	0.9936
추가 후	10729	118	0.9914	**0.9811**	**0.9948**

*성능지표는 지표마다의 Best Epoch 기준, 각 모델 튜닝 후 최고 성능지표

위의 표에서 확인할 수 있다시피 클래스 수는 2배 이상 늘어났고, 대회 기준 성능지표인 mAP@75:95와, F1 Score가 유의미하게 상승한 것을 확인할 수 있었다.

결론 : 추가 데이터 + 하이퍼파라미터 튜닝을 거친 **YOLO11s** 모델이 가장 높은 **mAP@75:95**를 보였고, 이를 최종 제출 모델로 선정하고 어플리케이션에 활용하기로 결정했다.



6. UI 디자인 및 구현

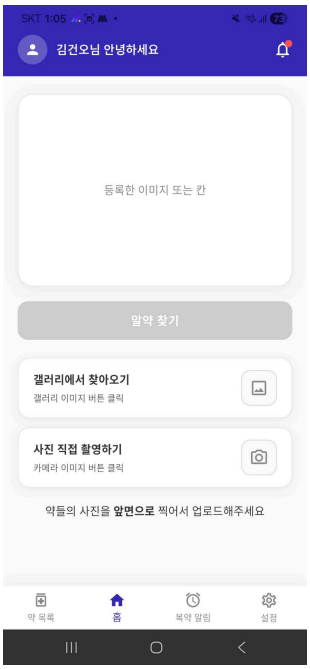
6.1 스플래시 스크린



- 앱 실행 시 2.5초간 표시되는 인트로 화면입니다.

- 페이드인, 슬라이드업, 웨이브 애니메이션이 순차 실행되며, 알약 아이콘과 웨이브 배경은 CustomPainter로 구현.

6.2 홈 스크린



- 약 사진을 갤러리 또는 카메라로 선택한 뒤 "알약 찾기" 버튼을 누르면 서버로 이미지를 전송해 분석을 요청

- 분석 중에는 로딩 스피너가 표시되며, 결과가 오면 결과 화면으로 자동 전환

6.3 알약 리스트 스크린



- 인식 가능한 전체 약품 목록을 서버에서 불러와 표시

- 검색 기능으로 약품명을 필터링할 수 있으며, 각 항목을 탭하면 효능·복용법·주의사항을 다이얼로그로 확인 가능.

6.4 복약 알림 관리



- 등록된 복약 알림 목록을 표시.

- 약품명, 복용 시간, 복용량, 메모를 확인할 수 있으며, 카드를 왼쪽으로 밀면 알림이 삭제됨

6.5 환경설정



- 프로필 사진과 이름을 변경할 수 있으며, 슬라이더로 앱 전체 폰트 크기를 조절 가능, 변경된 설정은 SharedPreferences에 저장되어 앱 재실행 후에도 유지.